

Build Automation

Setup & Configuration

User Manual

aiChair Backend

Version 1

WWU CS Senior Project 2

Spring 2026

Table of Contents

1	Revision History	3
2	System Overview	3
2.1	What gets built	3
2.2	System diagram	3
3	Build Orchestration with Docker Compose	4
3.1	The compose file	4
3.2	Compose topology.....	5
3.3	Running the build	5
3.4	Verifying the build	6
4	Build Scripts (Dockerfiles).....	6
4.1	api/Dockerfile	6
5	Dependency Management with pip-tools	7
5.1	Adding or updating a dependency	7
Appendix A	First-Time Setup	7
A.1	Clone the repo.....	7
A.2	Set up the .env file	7
A.3	First run	7

1 Revision History

Rev.	Date	Authors	Comments
1	5/23/2026	Kai Ketcham	Initial release.

2 System Overview

This document covers the build automation for the aiChair backend, a REST API service that supports an accessibility-focused routing app for powered wheelchair users in Bellingham. The backend is two containerized services that get built and run together with Docker Compose.

2.1 What gets built

The build produces two Docker images:

- api: the FastAPI service that exposes the REST endpoints /health, /obstacles, and /route.
- unweaver: the routing engine that computes shortest paths with accessibility cost profiles.

Each image is built from its own Dockerfile, and the two are wired together by docker-compose.yml at the repo root.

2.2 System diagram

The backend sits between a cloud-hosted database (Supabase) and the frontend. FastAPI is the central piece: it serves the frontend, calls Unweaver for routing, reads obstacles from the database when computing a route, and writes new obstacles when one is reported.

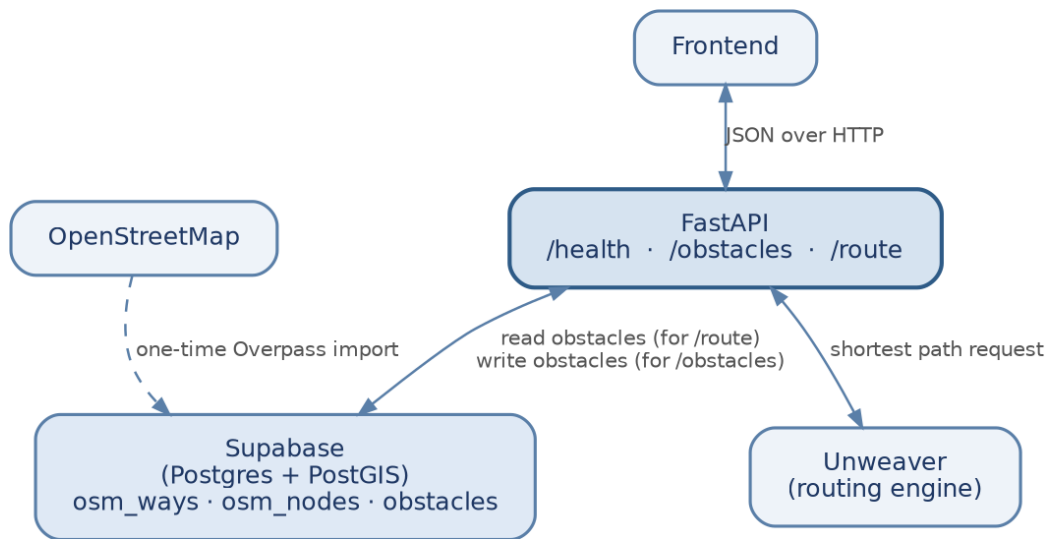


Figure 1. Data flow through the backend. The dashed line marks the one-time OSM import, which happens outside of normal runtime.

3 Build Orchestration with Docker Compose

Docker Compose is the orchestration tool for this project. One command builds both images, sets up the internal network between them, wires environment variables in, and starts them. The same command works on a developer laptop and on the deployment VM.

3.1 The compose file

The compose file at the repo root describes both services and how they connect:

```

services:
  unweaver:
    build: ./unweaver
    container_name: aichair-unweaver

  api:
    build: ./api
    container_name: aichair-api
    ports:
      - "8001:8001"
    environment:
      DATABASE_URL: ${DATABASE_URL}
      UNWEAVER_URL: http://unweaver:8000
    depends_on:
      - unweaver
  
```

A few notes on what's happening here:

- Only the `api` service publishes a port to the host. Unweaver is reachable from inside the compose network at `http://unweaver:8000` but is not exposed to the outside. This matches the production topology, where only the API sits behind the reverse proxy and the routing engine stays internal.
- `DATABASE_URL` is pulled in from the local `.env` file. `UNWEAVER_URL` is hardcoded in the compose file on purpose, since inside the container network Unweaver lives at hostname `unweaver` rather than `localhost`, so the `.env` value would not be valid in this context.
- `depends_on` tells Compose to start Unweaver first. It does not block on a readiness check, but it does order the startup so logs come out in a predictable sequence.

3.2 Compose topology

The diagram below shows how the two services and their network ports relate to the host machine.

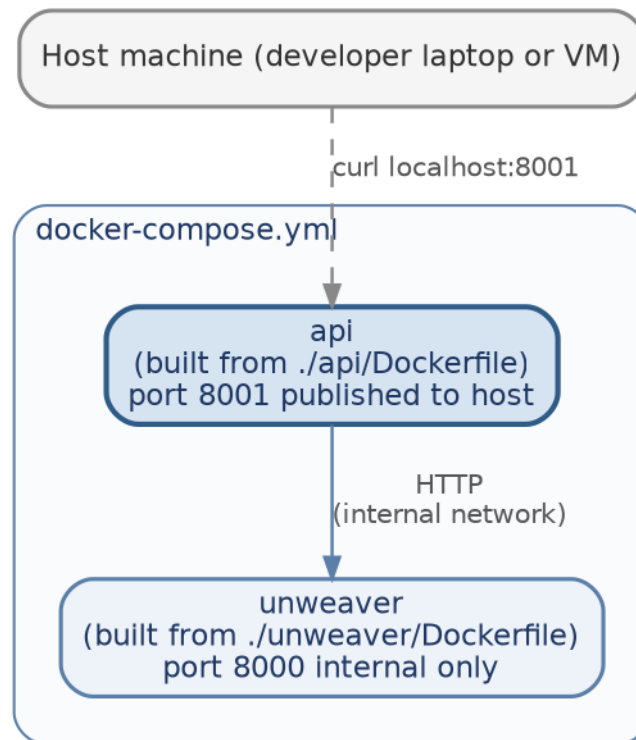


Figure 2. Compose topology. Only port 8001 is exposed to the host; Unweaver is internal-only.

3.3 Running the build

From the repo root, after cloning and copying `.env.example` to `.env` and filling in the Supabase password:

```
docker compose up --build
```

The `--build` flag forces both images to rebuild. Using `--build` every time avoids the common confusion of running unchanged images after editing source code.

To stop everything:

```
docker compose down
```

3.4 Verifying the build

Once the containers are up, the API responds on port 8001. The simplest sanity check is the health endpoint:

```
curl http://localhost:8001/health
```

Expected output:

```
{"status": "ok"}
```

A fuller end-to-end smoke test is a route request, which exercises the full path from the API to Unweaver and back through Supabase for nearby obstacles:

```
curl -X POST http://localhost:8001/route \
-H "Content-Type: application/json" \
-d '{"lat1":48.75,"lon1":-122.48,"lat2":48.76,"lon2":-122.47,"profile":"wheelchair"}'
```

If the response is a JSON object with segments and obstacles fields, the build is working end to end.

4 Build Scripts (Dockerfiles)

Each service has its own Dockerfile describing how to build its image. This section documents the FastAPI build script; Unweaver has its own Dockerfile maintained alongside the routing engine itself.

4.1 api/Dockerfile

```
FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

EXPOSE 8001

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8001"]
```

The order of operations matters. Requirements are copied and installed first, then the source code is copied in. Docker caches each layer, so when only main.py changes, the dependency install layer is reused instead of being redone. This keeps incremental rebuilds fast.

The --host 0.0.0.0 argument binds uvicorn to all network interfaces inside the container. This is necessary because the container's network is isolated from the host except through the published port; binding only to localhost inside the container would make the API unreachable from outside.

5 Dependency Management with pip-tools

The FastAPI service uses pip-tools for pinned dependencies. The pattern is straightforward:

- requirements.in: the short list of top-level packages the service actually depends on (fastapi, uvicorn, asyncpg, httpx).
- requirements.txt: the fully resolved, version-pinned dependency tree, generated by running pip-compile against the .in file.

Only requirements.txt is installed in the Docker image. The .in file is edited by hand; the .txt file is generated.

5.1 Adding or updating a dependency

```
# 1. add the package name to api/requirements.in
# 2. regenerate the lockfile
pip-compile api/requirements.in
# 3. rebuild the image
docker compose up --build api
```

pip-tools was chosen over running pip freeze because pip freeze tends to pollute the lockfile with whatever else happens to be installed in the local virtual environment. With pip-tools, the .in file is the source of truth for declared dependencies, and the .txt file is reproducible from it.

Appendix A First-Time Setup

These are the steps specific to this project. Docker, Docker Compose, and Git are assumed to be installed.

A.1 Clone the repo

```
git clone https://github.com/ketchkai/aichair-backend.git
cd aichair-backend
```

A.2 Set up the .env file

Copy .env.example to .env (which is gitignored) and fill in the real values:

```
cp .env.example .env
```

The two variables needed:

- DATABASE_URL: the Supabase session-pooler connection string. Find it in the Supabase dashboard under Connect, then Connection string, then Session pooler. Replace [YOUR-PASSWORD] with the database password (not the dashboard login password).
- UNWEAVER_URL: overridden by the compose file at runtime, so the default in .env.example can be left as-is.

A.3 First run

```
docker compose up --build
```

The first run takes a few minutes because Docker has to pull base images and install all dependencies fresh. After that, rebuilds are mostly cached and complete in seconds.